

Medium

Search

Write

Sign up

Sign in



★ Member-only story

Agent2Agent (A2A) Protocol in C#



Damien Vande Kerckhove

Follow

13 min read · Aug 20, 2025



1

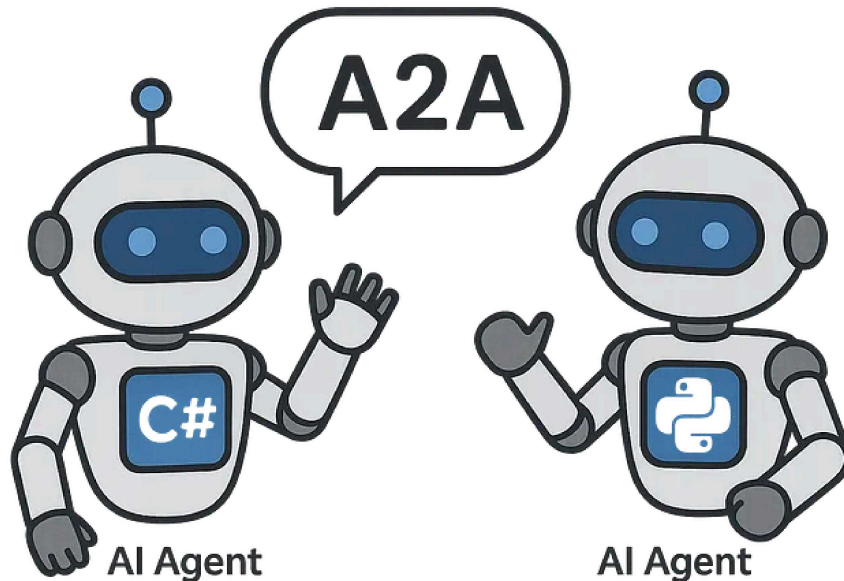


Introduction

If you've been following the world of AI, even a little, you've likely noticed that 2025 is becoming "the year of AI agents". We're seeing a surge of specialized agents, each focused on a specific task. In architectures that involve multiple AI agents, they can be compared to microservices.

Just like microservices communicate through standardized protocols (such as HTTP or gRPC), our AI agents also need a common way to interact. That's where the A2A protocol comes in!

The Agent2Agent (A2A) protocol is an open standard designed to enable AI agents to communicate and collaborate across different platforms and frameworks, regardless of their underlying technologies. Introduced by Google in April 2025, A2A addresses a fundamental challenge: how to enable autonomous agents, built by different teams using diverse technologies, to work together effectively. In this article, we'll show you how to use A2A in C# . Get ready to unlock seamless AI integration with just a few lines of code!



Getting started with a basic Echo Agent

Agents expose a public **Agent Card**, a JSON metadata file available at `/.well-known/agent.json`, which includes the agent's name, what it can do, contact endpoint, and how to interact with it. This is a core concept of the **A2A framework**, serving as the agent's identity card in the ecosystem.

```
{
  "name": "Echo Agent Demo",
  "description": "A simple demonstration agent that echoes back any message it r",
  "url": "http://localhost:5000/agent",
  "provider": { "organization": "DamienVDK" },
  "version": "1.0.0",
  "protocolVersion": "0.2.6",
  "capabilities": {
    "streaming": true,
    "pushNotifications": false,
    "stateTransitionHistory": false,
    "extensions": []
  },
  "skills": [],
}
```

We won't be writing this JSON manually. Instead, we'll generate it using the A2A .NET SDK. Let's start by creating our project and adding the SDK.

```
dotnet new webapp -n A2AAgent -f net9.0
cd A2AAgent
dotnet add package A2A --prerelease
dotnet add package A2A.AspNetCore --prerelease
```

We'll add a class called `EchoAgent` to represent our agent, along with a method that returns an `AgentCard` object. *Note: Although this method doesn't need to be asynchronous for now, we'll still use the `async/await` keywords. One of the core principles of the A2A framework is **Async First**.*

```
public class EchoAgent
{
    private async Task<AgentCard> GetAgentCardAsync(string agentUrl, Cancellatio
    {
        return await Task.FromResult<AgentCard>(new AgentCard()
        {
            Name = "Echo Agent Demo",
            Description = "A simple demonstration agent that echoes back any mes
            Url = agentUrl,
            Capabilities = new AgentCapabilities() { Streaming = true }
        });
    }
}
```

The next step is to register our agent with the A2A framework. In other words, make it available on the network so it can communicate with others. To do this, we'll simply add an `Attach` method that takes an `ITaskManager` instance as a parameter. The `ITaskManager` interface allows us to define which methods should be triggered in response to specific events. Let's start by linking our `GetAgentCardAsync` method so it returns the agent's Agent Card when requested.

```
public class EchoAgent
{
    public void Attach(ITaskManager taskManager)
    {
        taskManager.OnAgentCardQuery = GetAgentCardAsync;
    }

    private async Task<AgentCard> GetAgentCardAsync(string agentUrl, Cancellatio
    {
        return await Task.FromResult<AgentCard>(new AgentCard()
        {
            Name = "Echo Agent Demo",
            Description = "A simple demonstration agent that echoes back any mes
            Url = agentUrl,
            Capabilities = new AgentCapabilities() { Streaming = true },
            Skills =
```

```

        [
            new AgentSkill
            {
                Name = "Echo",
                Description = "Echoes back the received message"
            }
        ]
    });
}
}

```

To complete our first agent, we'll simply add a method to handle incoming messages. To keep this initial example straightforward, the agent will just reply with: **"Echo: [Received message]"**. This method will be linked to the `OnMessageReceived` event of the `ITaskManager`.

```

public class EchoAgent
{
    public void Attach(ITaskManager taskManager)
    {
        taskManager.OnMessageReceived = ProcessMessageAsync;
        taskManager.OnAgentCardQuery = GetAgentCardAsync;
    }

    private async Task<Message> ProcessMessageAsync(MessageSendParams messageSendParams)
    {
        var request = messageSendParams.Message.Parts.OfType<TextPart>().First();
        var response = $"Echo: {request}";

        return await Task.FromResult<Message>(new Message()
        {
            Role = MessageRole.Agent,
            Parts = [new TextPart() { Text = response }]
        });
    }

    private async Task<AgentCard> GetAgentCardAsync(string agentUrl, CancellationToken cancellationToken)
    {
        return await Task.FromResult<AgentCard>(new AgentCard()
        {
            Name = "Echo Agent Demo",
            Description = "A simple demonstration agent that echoes back any messages",
            Url = agentUrl,
            Capabilities = new AgentCapabilities() { Streaming = true },
            Skills =
            [
                new AgentSkill
                {
                    Name = "Echo",
                    Description = "Echoes back the received message"
                }
            ]
        });
    }
}

```

We'll take a closer look at the contents of this method shortly. But first, let's see how to test our agent. To do that, we need to update our `Program.cs` to specify the endpoint where the agent will be available.

```
using A2A;
using A2A.AspNetCore;

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

var agent = new EchoAgent();
var taskManager = new TaskManager();

agent.Attach(taskManager);

app.MapA2A(taskManager, "/agent");

Console.WriteLine("A2A Echo Agent is running on http://localhost:5000/agent");

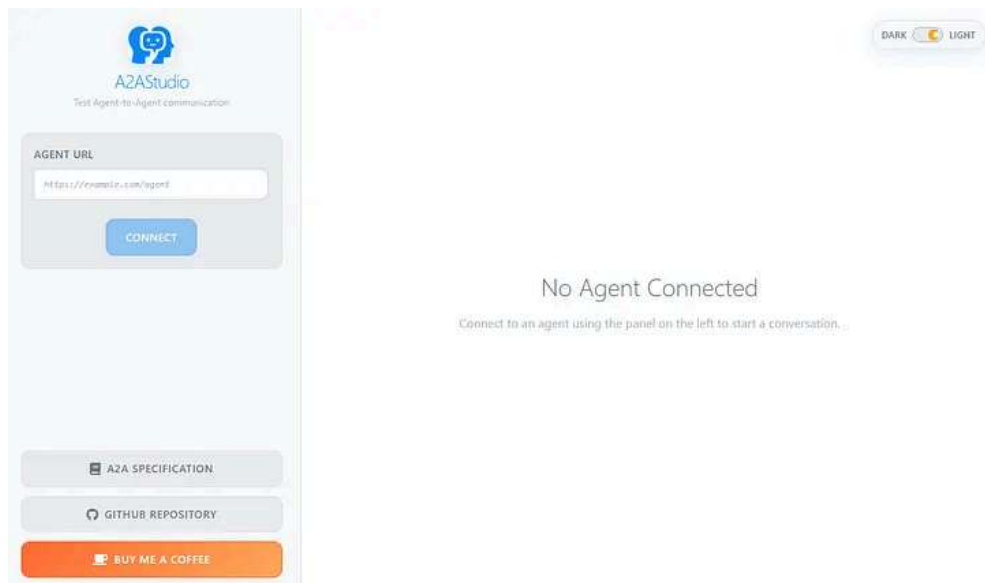
// Start the web app in the background
await app.RunAsync();
```

If you run your app and navigate to this URL: `http://localhost:5000/.well-known/agent.json`, you'll see the Agent Card in JSON format. However, we're not yet able to fully test our agent. While we could create a client using the A2A.NET SDK, for now we'll use a tool called **A2AStudio** to test our agent.

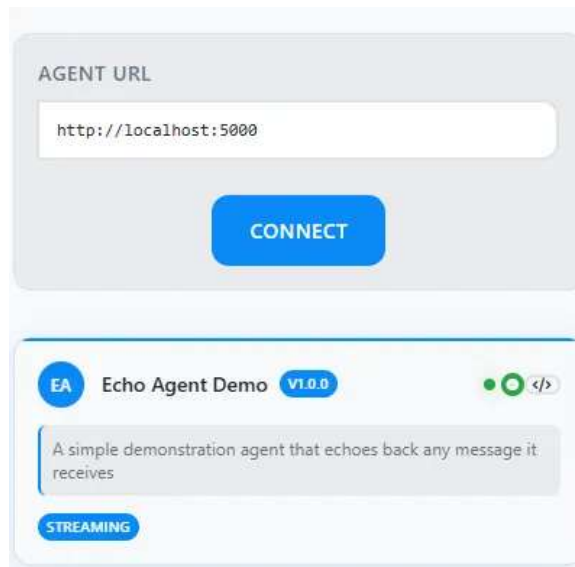
Exploring the A2AStudio Tool

A2AStudio is a web-based tool designed to help developers inspect, debug, and validate servers implementing Google's **A2A (Agent-to-Agent) protocol**. It provides an intuitive interface to interact with an A2A agent, monitor communications, and ensure compliance with the specification.

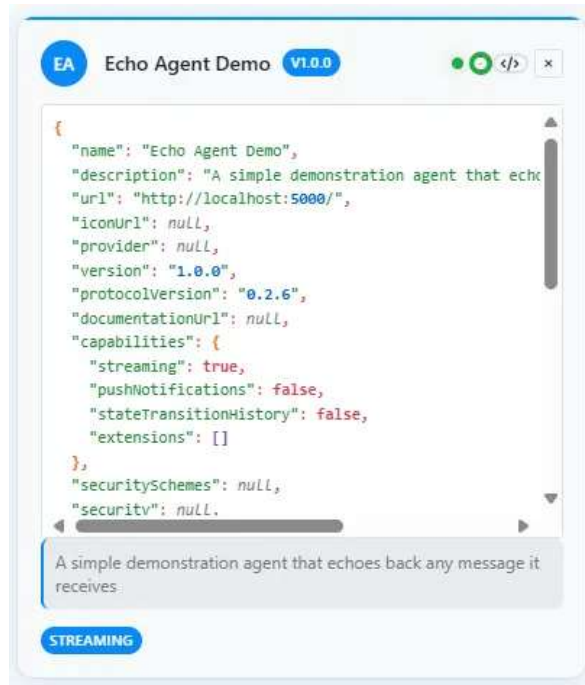
Since the application is built with WebAssembly, you can also run it directly against your local agent. The tool is available at [A2AStudio.net](https://a2astudio.net).



Enter your app's URL (e.g., `http://localhost:5000`) and click the **Connect** button. The Agent Card will be displayed, allowing you to verify its contents and check for any errors.



If you click the `</>` button, you can view the raw JSON representation of the agent card returned by the Agent.



Next, at the right of the app, you can send a message to your agent and view the response. If you set a breakpoint in the `ProcessMessageAsync` method, it will be triggered when the message is processed.



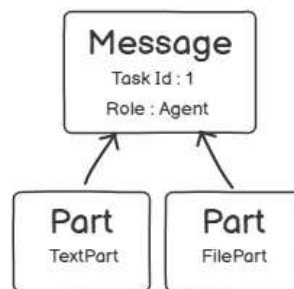
Protocol data objects : Part and Message

In the `EchoAgent.ProcessMessageAsync` method from our first example, we retrieve the received message using the following line:

```
messageSendParams.Message.Parts.OfType<TextPart>().First().Text.
```

A **Part** is the smallest unit of content within a message or an artifact. Each Part represents a specific type of data and must be either a `TextPart`, a `FilePart`, or a `DataPart`. A `TextPart` contains a single property called `Text`, which holds a plain string of text. A `FilePart` represents a file and includes a `File` property, which can take two forms: either a `FileWithBytes`, where the file content is embedded and encoded in Base64 using the `Bytes` property, or a `FileWithUri`, where the file is referenced by a URL available in the `Uri` property. A `DataPart` is used for structured data and includes a `Data` property, which is a dictionary mapping strings to JSON elements.

A **Message** represents one exchange in the conversation between a user and an agent. It always includes a `role` that indicates who sent the message—either "user" for client messages or "agent" for responses generated by the AI. Each message also contains one or more Parts, which together define its full content. These Parts can be of different types within the same message; for example, a message might include both a `TextPart` and a `FilePart`. In addition, every message includes a `TaskId`, which links it to the specific Task it belongs to.



Protocol data objects : Task

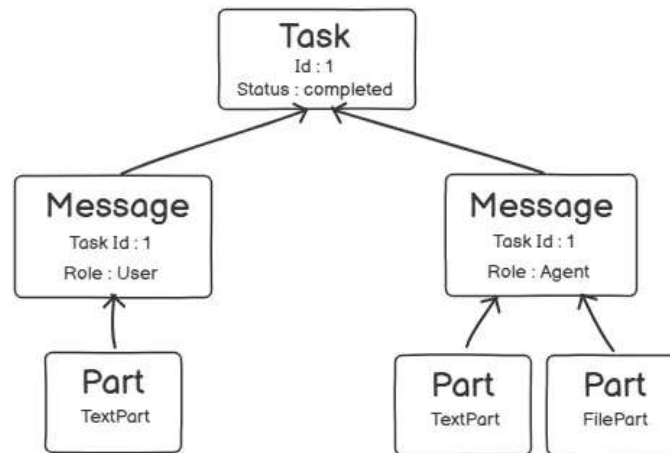
In the example shown earlier, the agent responds immediately to the received message, as the task is very simple to complete. However, in real-world scenarios, you will most likely need to build agents that handle more complex tasks, some of which may take time to process or require multiple steps.

Fortunately, the A2A framework is well-designed and includes a powerful concept called a **Task**. A Task is a stateful object that allows the client and the agent to collaborate effectively. It helps track the progress of a task, manage its state, and generate artifacts along the way.

A **Task** object contains several key elements: its **status**, the **history of exchanged messages**, and optionally a **context ID** (useful when multiple related tasks need to be grouped together).

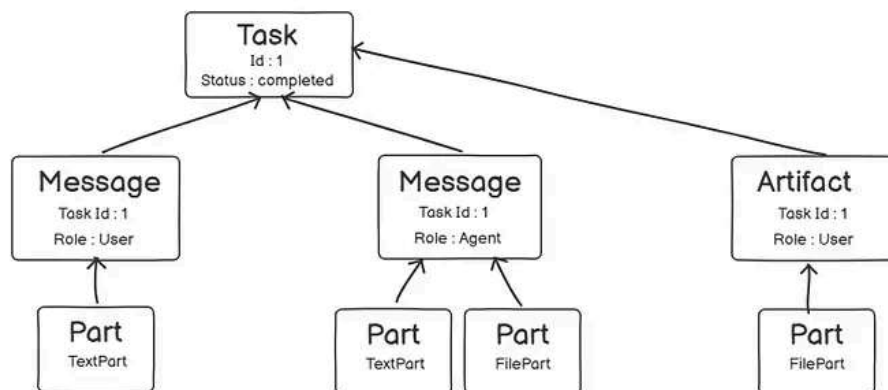
The status of a Task can be one of the following:

Submitted, Working, InputRequired, Completed, Canceled, Failed, Rejected, AuthRequired, or Unknown.



If needed, an agent can also generate one or more **artifacts** as output for a Task.

An **artifact** is a structured result produced by the agent. It includes a **name**, a **description**, and one or more **Parts** that represent the actual content, such as text, files, or structured data.



Extending the Echo Agent to support tasks

Now that we've learned about the concept of **Tasks**, let's see how we can update our agent to handle them properly.

To do this, we'll revisit the `Attach` method, which takes an instance of `ITaskManager` as a parameter.

```
public void Attach(ITaskManager taskManager)
{
    taskManager.OnMessageReceived = ProcessMessageAsync;
    taskManager.OnAgentCardQuery = GetAgentCardAsync;
}
```

The `ITaskManager` interface provides three additional methods that allow the agent to respond to task-related events: `OnTaskCreated`, `OnTaskUpdated`, and `OnTaskCancelled`.

Each of these methods shares the same signature: they receive an `AgentTask`, which represents the `Task` object from the A2A framework—and a `CancellationToken` as parameters.

These methods enable your agent to react appropriately when a task is created, updated, or cancelled during its lifecycle.

The `AgentTask` object contains the full history of exchanged messages through its `History` property.

Since we're working with a simple Echo agent, the history will contain only the message sent by the client. To retrieve this message, we can simply access the last item in the history using:

```
agentTaskParams.History!.Last()
```

Each message is made up of one or more **Parts**. For this example, we'll focus on extracting the most recent `TextPart`. This can be done as follows:

```
lastMessage.Parts.OfType<TextPart>().First().Text
```

This gives us the plain text content from the message, which the Echo agent can then respond to.

To keep things simple, our agent will return an **Artifact** containing two elements:

- The original message, prefixed with the word "Echo"
- An image, using a hardcoded URL.

This demonstrates how an agent can return structured output made of multiple Parts, such as text and media, grouped together in a single artifact.

```
await _taskManager!.ReturnArtifactAsync(agentTaskParams.Id, new Artifact
{
    Parts =
    [
        new TextPart
        {
            Text = $"Echo: {messageText}"
        },
        new FilePart
        { File = new FileWithUri { Uri = "https://s.w.org/images/core/emoji/"
    ]
}, cancellationToken);
```

Finally, we just need to update the task's status to `Completed` using the `UpdateStatusAsync` method provided by the `TaskManager`.

This method includes a final parameter that allows us to indicate whether the status is final. Setting this to `true` ensures that any open streams related to the task are properly closed.

Below is our updated version of the `EchoAgent` that reflects these changes.

```
public class EchoAgent
{
    private ITaskManager? _taskManager;

    public void Attach(ITaskManager taskManager)
    {
```

```

_taskManager = taskManager;
taskManager.OnAgentCardQuery = GetAgentCardAsync;
taskManager.OnTaskCreated = ProcessTaskCreatedAsync;
}

private async Task ProcessTaskCreatedAsync(AgentTask agentTaskParams, CancellationTok
{
    var lastMessage = agentTaskParams.History!.Last();
    var messageText = lastMessage.Parts.OfType<TextPart>().First().Text;

    await _taskManager!.ReturnArtifactAsync(agentTaskParams.Id, new Artifact
    {
        Parts =
        [
            new TextPart
            {
                Text = $"Echo: {messageText}"
            },
            new FilePart
            {
                File = new FileWithUri { Uri = "https://s.w.org/images/core
            }
        ], cancellationTokens);

    await _taskManager!.UpdateStatusAsync(
        agentTaskParams.Id,
        TaskState.Completed,
        final: true,
        cancellationTokens: cancellationTokens);
}

private async Task<AgentCard> GetAgentCardAsync(string agentUrl, CancellationTok
{
    // Same as before
}
}

```

You can now test the new version of our Echo Agent using the **A2A Inspector**. When you do, you should see a result similar to the following.



Let's now look at how we can enhance our agent by connecting it to Azure OpenAI in order to use an LLM model. This will allow us to see how a Task completion works when it requires more than one step to be fully completed.

If you want to follow this exercise exactly as shown, you'll need an account in Azure AI Foundry and a deployed model there. Of course, you can also connect your agent to another model, even if it's not hosted in Azure.

To begin, we'll add the necessary libraries so that our agent can connect to our LLM model in Azure AI Foundry. For this, we'll use the `Microsoft.AI.Hosting` library.

```
dotnet add package Microsoft.Extensions.AI
dotnet add package Microsoft.Extensions.AI.OpenAI
```

To connect our application to **Azure AI Foundry**, you will need :

- **The name of your deployed model**
- **The endpoint** for your Azure AI Foundry project
- **The API key** associated with that project

If you're not sure how to deploy a model in Azure AI Foundry, you can find the official guide here.

***Note:** In this example, we'll use the API key for simplicity. However, for better security in production, I strongly recommend using a **service principal** or even a **managed identity** if your environment supports it.*

You can store these values in one of the following ways: **AppSettings.json**, **Environment variables** or **User secrets**.

***Note:** If your project will be deployed to Azure, combining managed identities with Azure Key Vault is often the most secure approach.*

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  }
}
```

```

    },
    "AllowedHosts": "*",
    "AzureOpenAI": {
        "ChatDeploymentName": "gpt-4.1",
        "Endpoint": "https://YOUR-ENDPOINT.openai.azure.com",
        "ApiKey": "YOUR_API_KEY"
    }
}

```

We can now update the **Program.cs** file to connect our application to our model in **Azure AI Foundry**.

```

var builder = WebApplication.CreateBuilder(args);

var chatDeploymentName = builder.Configuration.GetValue<string>("AzureOpenAI:ChatDeploymentName");
var endpoint = builder.Configuration.GetValue<string>("AzureOpenAI:Endpoint");
var apiKey = builder.Configuration.GetValue<string>("AzureOpenAI:ApiKey");

builder.Services.AddSingleton<IChatClient>
(provider =>
    new ChatClientBuilder(
        new AzureOpenAIClient(new Uri(endpoint), new ApiKeyCredential(apiKey))
        .UseFunctionInvocation()
        .Build());

```

We will now create a new agent whose purpose is to return the current weather. The weather will only be returned if the user's domain includes a specified location.

Example: *What's the current weather in New York?*

If no location is provided, the Task will move to the **InputRequired** status and wait for another user interaction before continuing.

Let's begin by creating a new **WeatherAgent** class that returns an **AgentCard**.

```

using A2A;
using A2Agent.Services;
using Microsoft.Extensions.AI;

namespace A2Agent;

public class WeatherAgent
{
    private ITaskManager? _taskManager;

    public void Attach(ITaskManager taskManager)
    {
        _taskManager = taskManager;
    }
}

```

```

taskManager.OnAgentCardQuery = GetAgentCardAsync;
}

private async Task<AgentCard> GetAgentCardAsync(string agentUrl, Cancellatio
{
    return await Task.FromResult(new AgentCard
    {
        Name = "Weather Agent",
        Description = "An intelligent weather agent powered by Microsoft.Ext
        Url = agentUrl,
        Capabilities = new AgentCapabilities { Streaming = true },
        Skills =
        [
            new AgentSkill
            {
                Name = "Get Weather",
                Description = "Provides current weather information for a sp
            }
        ]
    });
}
}
}

```

In our earlier example with the Echo Agent responding to a Task, we saw that the `ITaskManager` interface allows us to handle the `OnTaskCreated` and `OnTaskCancelled` events.

We will now add a method that responds to both of these events.

As a first step, our method will check whether the request includes a location.

If not, the task will be updated with the status **InputRequired**.

```

private async Task ProcessTaskAsync(AgentTask agentTaskParams, CancellationToken
{
    var lastMessage = agentTaskParams.History!.Last();
    var messageText = lastMessage.Parts.OfType<TextPart>().First().Text;

    var locationCheck = await locationDetectionService.CheckForLocationAsync(mes

    if (!locationCheck.HasLocation)
    {
        await _taskManager!.UpdateStatusAsync(
            agentTaskParams.Id,
            TaskState.InputRequired,
            final: false,
            cancellationToken: cancellationToken);

        await _taskManager!.ReturnArtifactAsync(agentTaskParams.Id, new Artifact
        {
            Parts = [new TextPart { Text = "Please provide a location to get
        }, cancellationToken);

    }

    return;
}
}

```

```
}  
    // The rest of the method  
}
```

The next step is to return the weather when a location is provided. The LLM we are connected to, GPT-4.1, cannot fetch real-time data, and therefore cannot directly provide the current weather. To solve this, we will supply our model with an additional tool.

This tool (*function calling*) will be created using `AIFunctionFactory.Create` from the `Microsoft.Extensions.AI` library. The method expects a delegate as a parameter, in this case, our delegate will point to a method that returns the weather for the specified location.

Let's start by creating this method. You'll notice that both the method itself and its `location` parameter include a `Description` attribute. This attribute is important because it helps the LLM understand the purpose of the method and how it should be used.

```
public class WeatherService : IWeatherService  
{  
    [Description("Get the current weather for a specified location")]  
    public async Task<string> GetWeatherAsync([Description("The city or location")]  
    {  
        // Simulate API call delay  
        await Task.Delay(500);  
  
        var weatherData = GenerateMockWeatherData(location);  
        return FormatWeatherResponse(weatherData);  
    }  
  
    // The rest of the class  
}
```

In the final step, we simply need to call our LLM model and indicate which tool should be used. This tool is defined within the `ChatOptions` object, which we pass to the `GetResponseAsync` method. Once the response is received, the `Task` can then be updated to the **Completed** status.


```

public void Attach(ITaskManager taskManager)
{
    _taskManager = taskManager;
    taskManager.OnAgentCardQuery = GetAgentCardAsync;
    taskManager.OnTaskCreated = ProcessTaskAsync;
    taskManager.OnTaskUpdated = ProcessTaskAsync;
}

private async Task ProcessTaskAsync(AgentTask agentTaskParams, CancellationToken
{
    var lastMessage = agentTaskParams.History!.Last();
    var messageText = lastMessage.Parts.OfType<TextPart>().First().Text;

    var locationCheck = await locationDetectionService.CheckForLocationAsync(mes

    if (!locationCheck.HasLocation)
    {
        await _taskManager!.UpdateStatusAsync(
            agentTaskParams.Id,
            TaskState.InputRequired,
            final: false,
            cancellationTokens: cancellationTokens);

        await _taskManager!.ReturnArtifactAsync(agentTaskParams.Id, new Artifact
        {
            Parts =
            [
                new TextPart
                {
                    Text =
                        "Please provide a location to get the weather informatio

                }
            ]
        }, cancellationTokens);

        return;
    }

    var messages = new List<ChatMessage>
    {
        new(ChatRole.System, "You are a helpful weather assistant. Use the GetWe
        new(ChatRole.User, messageText)
    };

    var chatOptions = new ChatOptions
    {
        Tools = [AIFunctionFactory.Create(weatherService.GetWeatherAsync)],
        ToolMode = ChatToolMode.Auto
    };

    var response = await chatClient.GetResponseAsync(messages, chatOptions, canc

    await _taskManager!.UpdateStatusAsync(
        agentTaskParams.Id,
        TaskState.Completed,
        final: true,
        cancellationTokens: cancellationTokens);

    await _taskManager!.ReturnArtifactAsync(agentTaskParams.Id, new Artifact
    {
        Parts = [new TextPart { Text = response.Text }]
    }, cancellationTokens);
}

```

Push notification

This section is still being written. Stay connected and subscribe so you'll be the first to read it when it's ready!

Conclusion

In this article, we walked through the journey from a simple Echo Agent to a Weather Agent powered by **Azure OpenAI**, all built on top of the A2A framework. Along the way, we saw how A2A helps agents describe themselves, manage tasks, and collaborate with external services like LLMs, making it much easier to design scalable and interoperable multi-agent systems.

Now it's your turn to experiment, extend, and create your own agents. 🍷
Happy coding, and don't forget to share this article if you found it useful!

A2a

A2a Protocol

Ia Agents

Ia Agentic



1

**Written by Damien Vande Kerckhove**

46 followers · 8 following

Passionate developer with more than 15y of xp, I specialized in Microsoft technologies, mainly on .Net programming and the Azure environment.

Follow

**No responses yet**



Write a response

What are your thoughts?

More from Damien Vande Kerckhove



Damien Vande Kerckhove

Why You Shouldn't Call BuildServiceProvider in .NET...

Each invocation of BuildServiceProvider generates a new dependency container,...



Jan 22, 2024



40



Damien Vande Kerckhove

Securing Access to Blob Storage: Authentication and Best Practices

Since 2009, Microsoft's Azure cloud platform has provided the ability to store blobs in a...



Mar 31



Damien Vande Kerckhove

Experienced Story: C# — Checks for sufficient memory resources...

In my opinion, experiences feedback often make for the best talks and articles. That's...



Nov 26, 2023



Damien Vande Kerckhove

Master dependency injection in C# .Net

Introduction



Sep 8, 2023



77



See all from Damien Vande Kerckhove

Recommended from Medium

 In Byte-Sized AI by Don Moon

Mem0: Building Production-Ready AI Agents with Scalable Long-Ter...

Traditional LLMs are constrained by fixed context windows, making it difficult to...

★ Aug 31 🖱 14



 In Amex GBT technology by Aneshka Goyal

Introduction to Agent To Agent (A2A) protocol

What is A2A?

Oct 10 🖱 18



 In DataFrens.sg by Black_Raven (James Ng)

Google Gemini MedGemma

MedGemma & Vertex AI Model Garden for Healthcare

★ Aug 7 🖱 538 💬 1



 David Panonce

Building AI Agents with Google's ADK: Part 4—Why Your AI Agent...

“State is what makes an agent coherent. Memory is what makes it intelligent.”

Oct 5 🖱 2 💬 1



 Yash Bhaskar

MCP's Three Core Capabilities: Tools, Resources, and Prompts

Connecting an AI to external software is the key to building powerful applications. But...

 AshokReddy

MCP vs API: The Evolution That's Changing How Machines Really...

APIs built the modern web. MCP might redefine it for the age of AI. Here's why every...

★ Jun 30 🖱 7



★ 4d ago 🖱 13 💬 1



See more recommendations

[Help](#) [Status](#) [About](#) [Careers](#) [Press](#) [Blog](#) [Privacy](#) [Rules](#) [Terms](#) [Text to speech](#)